RX / SPORTS ANALYTICS

UO
Implementation Guide

NBA and NFL player opportunity models
From source observations to market probabilities

LOOKBACK / NEW SEASONS / CODE LOGIC / PRICING

A guide to the implementation in this repository, including the exact history windows, minimum samples, seasonal carryover, chronological calibration, saved forecast behavior and limitations.

Read section 0 first for the answers to how many games are used and what happens when a new season has little or no history.

The original research PDF is retained separately as a source reference. This manual explains what the code actually implements; advanced proposals in the source papers are not represented as completed features.

Code review date: 20 October 2026
Documentation source: models/Player_Stats/Usage_Opportunity/README.md

Contents

0. Lookback and new seasons 3

1. Source-to-model pipeline 3

2. Opportunity scenarios and allocation 4

3. Efficiency and uncertainty 4

4. NBA generation 5

5. NFL generation 5

6. Distribution, prices and criteria 6

7. Reproducibility, preparation and CLI 6

8. Validation and historical replay 6

9. Boundaries and future inputs 7

This folder implements an NBA/NFL box-score version of the supplied `Usage_Opportunity_Modeling_NBA_NFL_NCAAF.pdf`. The source PDF is preserved unchanged; this guide describes the actual code, assumptions and limits. The separate `UO_Implementation_Guide.pdf` is generated from this file.

The shared simulation functions accept an optional component-adjustment context used by the separate Opponent-Adjusted Matchup model. Plain UO supplies no such context, so its calculations and forecast assumptions remain unchanged. Optional environment hooks support the separate Pace & Game Environment strategy's conditioned role sampling, shared volume and overtime budgets. Plain UO supplies no environment, retaining its original regulation-only calculation path.

0. Lookback and new seasons

Setting	NBA	NFL
Recent player role window	Last 20 complete appearances	Last 16 complete appearances
Opportunity half-life	5 appearances	3 appearances
Efficiency history	Last 60 complete appearances	Last 32 complete appearances
Efficiency half-life	20 appearances	8 appearances
Team scenario history	Last 20 complete team games	Last 16 complete team games
Minimum complete player/team games	5 each	3 each
Simulation count	12,000	12,000

These are fixed research defaults in `config.py`, not empirically tuned settings. Each feature family must meet its own minimum; the effective intersection of required observed columns can be smaller than the total number of player games. Missing observations are dropped for that family, never converted to zero.

The repository reads the matchup season and the immediately previous season, including saved regular and postseason rows. NBA seasons use starting years; October starts the NBA lookup season. Football lookup seasons change in March, keeping January/February in the preceding football season. No season reset is applied inside an appearance window. Early in a new season, recent appearances from the previous season naturally supply the remaining window. Efficiency uses its longer window even when role history is short.

A new player without the minimum complete appearances has no forecast. No invented rookie baseline or league-average production is substituted. For a trade, identity is resolved using the latest known team across all saved clubs; only appearances with the player's current club enter the opportunity model. Consequently a recently traded player can remain unavailable until enough same-club history exists. Histories whose latest player observation is more than 450 days before the cutoff are rejected by the shared identity rules.

1. Source-to-model pipeline

`historical_data.py` reads the existing SQLite cache through `DataService` and `PlayerHistoryRepository`. NBA uses the application's NBA Stats/SportsDataverse adapters; NFL uses nflverse weekly team/player releases and the saved schedule. UO requires both actual team box scores and player appearances. Team production is never reconstructed by adding the sportsbook's quoted players.

Every forecast is cut off at the earlier of the snapshot pull time and game start. Both `event_date` and `available_at` must be strictly earlier than this cutoff. Rows with missing dates or availability preceding the game start are excluded. If publication timestamps are absent, the existing shared adapter uses eight hours after an explicit start, or the following

UTC day for date-only records. This is a conservative result-time proxy, not a point-in-time revision archive. Future rows are removed before fitting and before calculating the cache hash.

Identities require an exact normalized player name, a unique saved player ID and the latest known team belonging to the matchup. Ambiguous names are unavailable. Player/team opportunity ratios join on exact game ID and club; conflicting duplicate records fail rather than being arbitrarily selected.

2. Opportunity scenarios and allocation

`opportunity_model.py` computes normalized exponential recency weights:

```
w_i = 2^(-age_in_appearances / half_life)
normalized_w_i = w_i / sum(w)
```

The model samples a historical team row as a joint volume scenario. A common uniform quantile links the two matchup teams' scenarios. This provides shared positive dependence as an explicit research assumption; it is not an estimated opponent or game-script regression. Each team's selected row keeps its observed mix of shots or passing/rushing workloads.

A player's role is a recency-weighted draw of a complete historical appearance aligned with that game's team totals. Shot/carry/target shares from the same appearance move together. Role draws are conditional only on saved history, not on the held-out game's eventual minutes, snaps or touches.

Finite team pools are allocated by multinomial draws to all modeled players and an explicit Other bucket. If simulated shares exceed 100%, they are normalized. If they total less than 100%, Other receives the remaining probability. Counts sum exactly to the simulated team volume. Players absent from the sportsbook are represented by Other; the quoted players are not treated as the full roster.

3. Efficiency and uncertainty

`efficiency_model.py` uses Beta posteriors for NBA shooting, NFL catch rates and interceptions conditional on incomplete attempts. For a binary conversion:

```
alpha = alpha_0 + sum(power_weight_i * successes_i)
beta  = beta_0  + sum(power_weight_i * (trials_i - successes_i))
p_s ~ Beta(alpha, beta)
made_s ~ Binomial(attempts_s, p_s)
```

Defaults are Jeffreys Beta(0.5, 0.5). Normalized efficiency recency weights are multiplied by the number of retained games, giving a fractional power likelihood on event counts. This retains prior strength in units of events rather than normalizing an entire career to a single trial. The low-level function accepts explicit priors: the source example Beta(14,6) plus 18 catches in 20 targets gives posterior mean 32/40 = 0.80. That illustrative prior is not silently imposed on every player's catch rate.

Yards per event use a ratio of weighted totals, not an unweighted mean of per-game ratios. Conditional residuals are computed from observed game totals:

```
rate = sum(w_i * yards_i) / sum(w_i * events_i)
residual_i = (yards_i - rate * events_i) / sqrt(events_i)
predicted_yards_s = round(events_s * rate + sqrt(events_s) * sampled_residual_s)
```

Residuals are weighted-centered and resampled. Zero-event simulations produce zero yards; negative net yardage is allowed. This preserves observed game-level asymmetry and explosive-game residuals, but is not an empirical per-play gain distribution. The model has no target-depth, air-yard or YAC event feed. A player with only observed zero-event games has zero output for that channel; missing counts remain unavailable.

4. NBA generation

`nba_model.py` separates two-point attempts, three-point attempts and free throws. For each simulation it draws player minutes and related shares from a common player appearance, normalizes historical overtime minutes to regulation, and allocates finite team shot pools. A player is bounded to 48 regulation minutes. If quoted players exceed 240 total minutes, their minutes and allocation shares are scaled together. Other receives the remaining regulation minutes. The model does not forecast overtime or add five-minute overtime scenarios.

Two-point makes, three-point makes and free throws use their separate posterior conversion draws. Points are computed as `2*2PM + 3*3PM + FTM`, not by labeling usage rate as field-goal attempt share. For the PDF's fixed illustration, 10 two-point attempts at 55%, seven threes at 38% and six free throws at 85% yield an expectation of 24.08 points.

Rebound, assist, steal, block and turnover channels use saved team box-score pools and player allocation shares. Offensive and defensive rebounds are allocated separately. Assist volume is capped at generated team field goals; a player's assists cannot exceed field goals made by teammates. Removed assists return to Other. These channels are useful workload proxies, not measured rebound chances, potential assists, eligible steal events or defensive tracking. They do not enforce a full cross-team missed-shot/rebound event simulation.

The same player simulations generate points, rebounds, assists, threes, blocks, steals and turnovers. PA, PR, RA and PRA sum the underlying columns within each simulation, preserving their shared volume/minutes dependence. The engine does not multiply independent marginal prop probabilities to form combinations.

5. NFL generation

`nfl_model.py` uses team attempts, carries and sacks as a consistent observed offensive-play accounting baseline. Historical carries already include recorded QB scrambles and kneel-downs. Dropback probability is `(attempts + sacks)/(attempts + carries + sacks)`. It samples dropbacks, then sacks, leaving attempts; remaining eligible plays are carries.

Team targets are bounded by attempts using the historical target/attempt rate. Player target shares replace unavailable route participation times targets per route. Receiver targets are allocated across the finite pool; catches use player Beta-binomial conversion and receiving yards use the conditional residual model. Carries are independently allocated from the run pool and become rushing yards through the corresponding player efficiency fit.

The PDF's route illustration, 40 dropbacks times 0.90 route participation times 0.25 targets per route, yields nine targets; 68% catching and 12.5 yards per catch yield 6.12 catches and 76.50 yards. UO does not pretend those route inputs exist in weekly box scores. Its observed target-share approximation is labeled as such.

QB attempts share the team attempt pool. Generated receiver catches, including Other, form the exact team completion total. Hypergeometric allocation assigns these completions to QB attempts without any passer exceeding his attempts. Generated receiver yards form team passing yards; completions allocate this total to passers, with integer rounding assigned to Other. Thus receiver and QB outputs share real simulation states. This assumes a team receiving-yardage model for the passing offense; it does not independently estimate a replacement QB's depth-specific skill. Interceptions are drawn from incomplete attempts and cannot overlap completions.

Rushing/receiving/passing yards, pass attempts/completions/interceptions, carry attempts and receptions are supported. TD counts, anytime TD and first/last scorer are unavailable in UO. The source requires explicit red-zone or goal-line opportunities; those inputs and a scorer-order model are not connected. No touchdown probabilities are fabricated from an unrelated mean.

6. Distribution, prices and criteria

`distributions.py` stores the empirical probability mass of 12,000 shared samples. It returns the mean, median, standard deviation and 5th/95th percentiles, plus line-specific win, loss and push probabilities. Degenerate all-zero outcomes are valid distributions; an artificial variance floor is not added.

Fair-price probability is `win/(win + loss)` when a bet can settle. Whole-number pushes remain a separate refund probability. Fair American odds are computed by the shared market pricing code. Raw market implied probability retains the bookmaker margin; paired opposite quotes can also produce a no-vig probability in the stored evaluation record. Edge is fair-price probability minus raw market implied probability, in percentage points. EV per dollar is `P(win)*net_payout - P(loss)`, so pushes contribute neither a win nor a loss.

In FanDuel NBA/NFL games, select Usage & Opportunity in Player Forecast Method. Player tables then show UO probability, UO fair odds and the corresponding Edge. Apply Criteria uses this method for every calculated selection in the displayed player category, retaining existing match/no-match highlighting and Show more. Game Outcome Forecast Method remains independent; WHP remains the player default. The last input date, workload projections, predictive interval, Monte Carlo standard error and EV are available in the forecast hover/details.

7. Reproducibility, preparation and CLI

`prediction_service.py` simulates every supported quoted player in the full saved game once. Switching category keeps the same shared outcomes. A deterministic sport/game/config seed provides common random numbers across refreshes. Cache identity includes model version, implementation hashes, configuration, exact eligible input data, cutoff and saved quote contents. A source change or code change invalidates the cache; future rows cannot affect a historical fit.

Histograms, diagnostics, unsupported-market reasons and per-quote evaluations are atomically persisted under the ignored `data/modeling/player_stats/usage_opportunity/forecasts/` tree. Original snapshot CSVs and their API-credit records are not modified. Viewing or changing methods makes no HTTP request. If source history is missing, estimates remain unavailable.

```
python -m models.Player_Stats.Usage_Opportunity --sport NBA --prepare --start "2026-10-25T23:00:00Z"
python -m models.Player_Stats.Usage_Opportunity --sport NFL --prepare --start "2026-10-25T17:00:00Z"
python -m models.Player_Stats.Usage_Opportunity --sport NBA --snapshot
    "data/market_data/your_saved_snapshot.csv"
```

Preparation explicitly downloads player and team rows for both seasons, reusing existing caches where possible. `--refresh` requests source refreshes. Neither command pulls Odds API markets. `--simulations` changes the simulation count for pricing/replay; permitted counts are 1,000 through 200,000. Source downloads use existing public adapters; no new credentials or Python dependencies are required.

8. Validation and historical replay

`backtesting.py` replays an explicitly supplied JSON archive of canonical pregame snapshots against an actuals CSV. Actuals contain `game_id, subject, market, value`. The market key may include the alternate suffix. Snapshot pull time must precede game start. The service refits using only history available before that snapshot; actual target-game opportunities are never supplied as model predictors.

```
python -m models.Player_Stats.Usage_Opportunity --sport NFL --backtest-snapshots "archive.json"
    --actuals "actuals.csv"
```

`calibration.py` reports target MAE, RMSE, bias, 90% interval coverage, settled-quote Brier/reliability bins and three-outcome Brier score including pushes. Multiple quotes do not repeat a player's target error: the last forecast per game/player/ market is used for target MAE; quote diagnostics include all supplied quotes. All unavailable snapshots are recorded. Fixed research defaults are not tuned on the replay interval. No trained probability calibration map is applied and no accuracy or profit claim is made from a compilation/browser check.

Offline regression checks cover the PDF's beta example, integer pushes, EV, minute/shot allocation, QB/receiver conservation, future/late-result exclusion, traded-away identity rejection, cache reuse, UI columns and historical replay. Current saved NBA/NFL market files are also exercised without external requests. These establish calculation and integration correctness, not predictive quality.

9. Boundaries and future inputs

The implemented scope is NBA/NFL team-volume and player-role box-score modeling. NCAAF is disabled for UO; WHP college forecasting remains available. The PDF's college scheme effects and NCAA sack convention require a separate future adapter. NBA tracking chances, NFL routes/snaps and red-zone events, roster/injury scenarios, learned opponent/script coefficients and fitted probability calibration remain future inputs. A dedicated analytics/scenario page suggested by the source is not part of this integration; the existing market workflow is used.

Build provenance

The following SHA-256 hashes identify the reviewed calculation/service files. They are documentation provenance, not a runtime cache identity.

models/Player_Stats/Usage_Opportunity/README.md
`708034964a6c43b1dc02c9d58a2bef54d7169e7f72093b7c254d27d290e9df4d`

models/Player_Stats/Usage_Opportunity/__init__.py
`ff0d2ee88e69a7828767bb536385ad1a130e534e47cf9a605ea7025a029745e0`

models/Player_Stats/Usage_Opportunity/__main__.py
`c15fb684e5590b22ab1f7f1e03294fea20a9dc2e64531274b2522cc8c22c9ac3`

models/Player_Stats/Usage_Opportunity/backtesting.py
`35545ef2c3391aa27578db1966ae378d0603a4a9fda8e8cc864c174bd5a63845`

models/Player_Stats/Usage_Opportunity/calibration.py
`14645523aa11a5e7d1cac0fb3168215d4b03cd9787c54de71893038bcf709f2c`

models/Player_Stats/Usage_Opportunity/config.py
`64626d5784422ca0f4ac742ecf487af91eb251109cf4fbe636d994989c362a40`

models/Player_Stats/Usage_Opportunity/distributions.py
`46a0a1b903348080016c37a482e4bfc92ffaf71a6a0ad9e72a8e59207f22a891`

models/Player_Stats/Usage_Opportunity/efficiency_model.py
`1fc93ef6b455ec782aa55f4db223fc3f7d96ddbda1e7042d0e0e40e220c075bb`

models/Player_Stats/Usage_Opportunity/errors.py
`41be921ece93c5b3037a1d0f7f9afdad849242d4b5cbcb866c7479d78f8542b4`

models/Player_Stats/Usage_Opportunity/historical_data.py
`4a5d2b774f95c02ef8e3c3058cf7b7344103fec31e3533ad56c36656ef0dc417`

models/Player_Stats/Usage_Opportunity/market_comparison.py
`86e35e1c70dd979366b7610879aa2ff1015bacff77b8330642273cda6459be44`

models/Player_Stats/Usage_Opportunity/markets.py
`4ff707184722c94de0231d86044d282632417efb86a756f6374d83d040a8c9a8`

models/Player_Stats/Usage_Opportunity/nba_model.py
`73f83385fd6a84a3af0ecfc87d2574d6294fa7a0b7401522d02f9d4f093e675c`

models/Player_Stats/Usage_Opportunity/nfl_model.py
`bddfc84bf37ae3b76f11dd259b516b1eb7bec9d922e0b4bb297dc32e6bcb977c`

models/Player_Stats/Usage_Opportunity/opportunity_model.py
`50228ce5d91077d2e9677cd4cc1267232032db64d312de6844f13d7e9677f682`

models/Player_Stats/Usage_Opportunity/prediction_service.py
`c5dbcebd5574e1a17ff22a36dd5502805033eeb36d1df3c2e7694af7abd930d9`

models/Player_Stats/Usage_Opportunity/simulation.py
`1752ada6773c7499c343e8293a62300129812a8c4388a32d238b68922f5c71d8`

sports_analytics/usage_opportunity_service.py
`7c2dc28534bc4f01572aca8eba1565a9678fe82bc37fcae4e31f5977620a16ea`